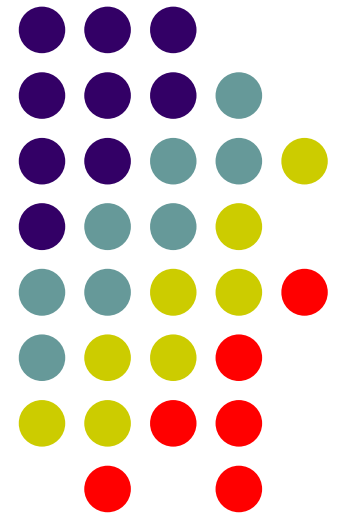
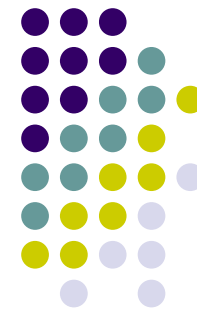


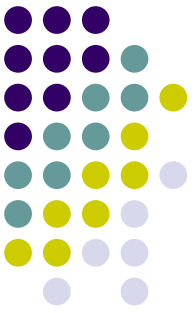
Ch5 Linux编程基础



本章内容

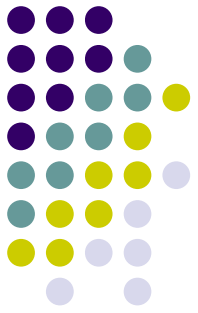
- GCC编译器
- GDB调试工具
- Make工具





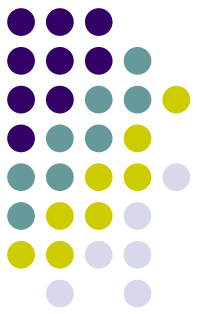
开发工具

- Vi/Vim, Gedit
- GCC
 - GNU C Compiler -> GNU Compiler Collection
 - The gcc command: Front end
- GDB
 - GNU Debugger
 - The gdb command
- Make



Part 1 GCC编译器

- GCC编译器介绍
- GCC编译过程
- GCC编译优化
- 头文件和库函数目录

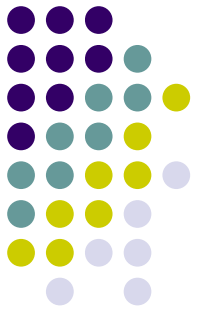


1.1 GCC编译器介绍

- GNU C Compiler -> GNU Compiler Collection

<http://gcc.gnu.org>

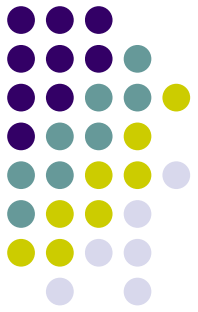
- GCC是一个强大的工具集合，它包含了预处理器、编译器、汇编器、链接器等组件。它会在需要的时候调用其他组件。
输入文件的类型和传递给gcc的参数决定了gcc调用具体的哪些组件。



1.1 GCC编译器介绍

GCC 参数选项

- Usage:
 - gcc [options] [filename]
- Basic options:
 - -E: 只对源程序进行预处理(调用cpp预处理器)
 - -S: 只对源程序进行预处理、编译
 - -c: 执行预处理、编译、汇编而不链接
 - -o output_file: 指定输出文件名
 - -g: 产生调试工具必需的符号信息
 - -O/On: 在程序编译、链接过程中进行优化处理
 - -Wall: 显示所有的警告信息
 - -I dir : 在头文件的搜索路径中添加dir目录
 - -L dir :在库文件的搜索路径列表中添加dir目录



1.2 GCC编译过程

一般情况下，C程序的编译过程为

1、预处理

2、编译成汇编代码

3、汇编成目标代码 *object*

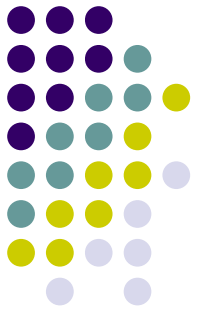
4、链接

预处理 .i

编译 → 汇编代码 .s

汇编 → 目标代码 .o

链接



1. 预处理

- 预处理：使用 -E 参数

-o 作用：
指定编译或链接过程
生成的文件的名称。

```
gcc -E -o gcctest.i gcctest.c
```

- 使用 **wc** 命令比较预处理后的文件与源文件，
可以看到两个文件的差异



```
[donger@donger gcctest]$ gcc -E -o gcctest.i gcctest.c
```

```
[donger@donger gcctest]$ ls
```

```
gcctest.c  gcctest.i
```

```
[donger@donger gcctest]$ wc gcctest.c gcctest.i
```

```
15      15    132 gcctest.c
```

```
929    2102 18137 gcctest.i
```

```
944    2117 18269 总用量
```

```
[donger@donger gcctest]$ wc --help
```

```
用法: wc [选项]... [文件]...
```

Print newline, word, and byte counts for each FILE, and a total line if

more than one FILE is specified. With no FILE, or when FILE is -,

read standard input.

-c, --bytes print the byte counts

-m, --chars print the character counts

-l, --lines print the newline counts

-L, --max-line-length print the length of the longest line

-w, --words print the word counts

--help 显示此帮助信息并离开

--version 显示版本信息并离开

Report bugs to <bug-coreutils@gnu.org>.

```
[donger@donger gcctest]$ █
```

行数 单词数 字节数



`.c`、`.i`、`.s`和`.o`文件分别是C程序编译过程中生成的不同文件类型：

1. `.c` 文件：这是C语言的源代码文件。它包含了你编写的C语言代码，是编译过程的起点。
2. `.i` 文件：这是预处理后的文件。C语言编译的第一步是预处理，预处理器会处理所有的宏定义、头文件包含和条件编译指令。预处理后的代码会被存储在`.i`文件中。可以通过以下命令生成`.i`文件：

```
bash
gcc -E source.c -o source.i
```

复制代码

3. `.s` 文件：这是汇编代码文件。预处理后的`.i`文件会被编译器编译成汇编代码，并存储在`.s`文件中。可以通过以下命令生成`.s`文件：

```
bash
gcc -S source.i -o source.s
```

复制代码

或者直接从`.c`文件生成：

```
bash
gcc -S source.c -o source.s
```

复制代码

4. `.o` 文件：这是目标文件。汇编代码会被汇编器转换成机器代码，存储在`.o`文件中。可以通过以下命令生成`.o`文件：

4. `.o` 文件：这是目标文件。汇编代码会被汇编器转换成机器代码，存储在`.o`文件中。可以通过以下命令生成`.o`文件：

```
bash
gcc -c source.s -o source.o
```

复制代码

或者直接从`.c`文件生成：

```
bash
gcc -c source.c -o source.o
```

复制代码

最后，`.o`文件可以与其他目标文件和库文件链接，生成最终的可执行文件。例如：

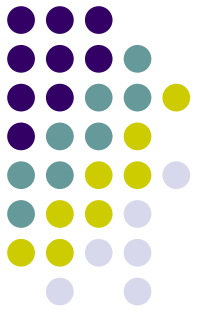
```
bash
gcc source.o -o executable
```

复制代码

总结来说，编译过程大致如下：

1. 源代码文件（`.c`） -> 预处理文件（`.i`）
2. 预处理文件（`.i`） -> 汇编代码文件（`.s`）
3. 汇编代码文件（`.s`） -> 目标文件（`.o`）
4. 目标文件（`.o`） -> 可执行文件

这些文件在不同的编译阶段分别生成，用于逐步将高级语言代码转换为机器可执行的二进制代码。



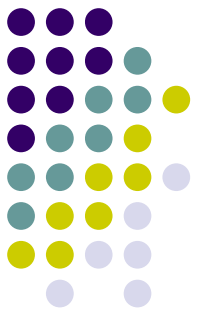
2.编译成汇编代码

- 预处理文件→汇编代码
使用-S说明生成汇编代码后停止工作

```
gcc -S -o gcctest.s gcctest.i
```

- 直接编译到汇编代码

```
gcc -S gcctest.c
```



3. 编译成目标代码

- 汇编代码 → 目标代码

`gcc -x assembler -c gcctest.s`

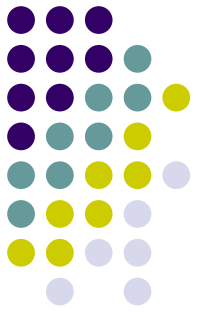
- 直接编译成目标代码

`gcc -c gcctest.c`

编译选项 `-c` 生成目标文件 `gcctest.o`, 但不进行链接。

- 使用汇编器生成目标代码

`as -o gcctest.o gcctest.s`



4. 编译成执行代码

- 目标代码 → 执行代码

`gcc -o gcctest gcctest.o`

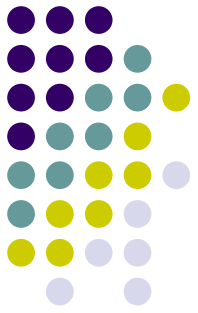
-o + 目标文件名

→ 可执行文件

- 直接生成执行代码

`gcc -o gcctest gcctest.c`

↓
可执行文件

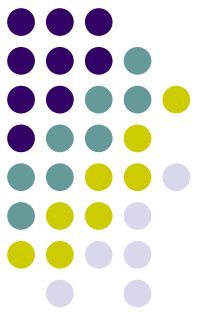


1.3 GCC编译优化

- 优化编译选项有：
 - -O0
缺省情况，不优化
 - -O1
 - -O2
 - -O3
 - 等等

gcc的优化编译举例（1）

考虑如下的源代码



```
//mytest.c
#include <stdio.h>
#include <math.h>

int main()
{
    int i,j;
    double k,k1,k2,k3;
    k=0.0; k1=k2=k3=1.0;
    for (i=0;i<50000;i++)
        for (j=0;j<50000;j++)
        {
            k+=k1+k2+k3;
            k1 += 0.5;
            k2 += 0.2;
            k3 = k1+k2;
            k3 -= 0.1;
        }
    return 0;
}
```

```
[donger@donger gcctest]$ ls
gcctest.c  mytest.c
[donger@donger gcctest]$ gcc -O0 -o m0 mytest.c
[donger@donger gcctest]$ gcc -O1 -o m1 mytest.c
[donger@donger gcctest]$ gcc -O2 -o m2 mytest.c
[donger@donger gcctest]$ gcc -O3 -o m3 mytest.c
[donger@donger gcctest]$ ls
gcctest.c  m0 m1 m2 m3 mytest.c
[donger@donger gcctest]$
```

不同的优化
编译选项

gcc的优化编译举例（2）

使用time命令统计程序的运行



```
[donger@donger gcctest]$ ls
gcctest.c  m0  m1  m2  m3  mytest.c
[donger@donger gcctest]$ time ./m3

real    0m2.756s
user    0m2.658s
sys     0m0.042s
[donger@donger gcctest]$ time ./m2

real    0m2.733s
user    0m2.643s
sys     0m0.037s
[donger@donger gcctest]$ time ./m1

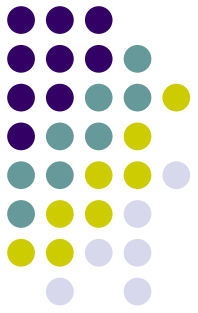
real    0m1.829s
user    0m1.767s
sys     0m0.022s
[donger@donger gcctest]$ time ./m0

real    0m40.808s
user    0m39.632s
sys     0m0.337s
[donger@donger gcctest]$
```

```
mylinux@ubuntu:~/program$ time ./m0

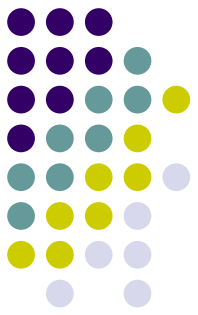
real    0m0.006s
user    0m0.005s
sys     0m0.000s
mylinux@ubuntu:~/program$ time ./m1

real    0m0.005s
user    0m0.001s
sys     0m0.004s
```

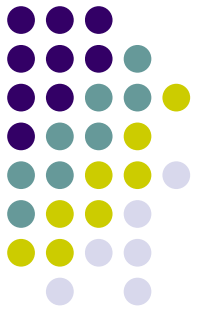
1.4 头文件和库函数目录

- GCC -I dir 参数使用
- GCC 创建库函数



1. GCC -I dir 参数使用

- 头文件和gcc不在同一目录下，用 -I dir 指明头文件所在的目录。
- `#include <>`: 在默认路径 “/usr/include” 中搜索头文件
- `#include ""`: 在 本目录中搜索
在源程序所在目录



Example

1-3.c //main程序

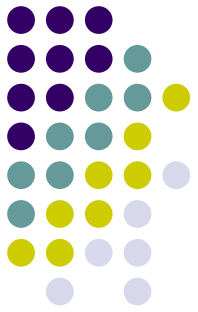
```
#include <my.h>    /*文件预处理，包含自定义的库文件"my.h"*/
int main()
{
    char ch;
    while((ch=getchar())!=EOF)    //ctrl+D
        putchar(ch); /
    return 1;
}
```

*my.h程序：自定义的头文件

```
#include <stdio.h>
```

```
/*文件预处理，包含标准输入输出库*/
```

运行结果？



大写的i

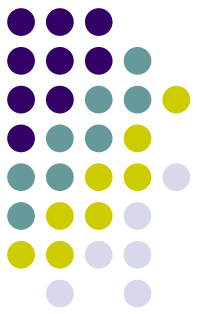
- 解决办法:

1. gcc opt.c -o opt -I ./ 当前目录

2. 修改main

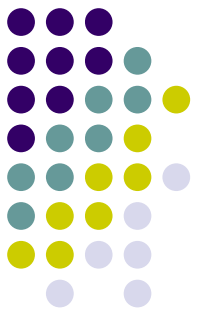
```
#include <my.h>
```

```
#include "my.h"
```



2. GCC创建函数库

- 函数库：公用函数定义为函数库，供其他程序使用。函数库分为静态库和动态库。
- ① 静态库：程序编译时会链接到目标代码中，程序运行时不再需要静态库。程序生成的可执行程序比较大。后缀名为“.a”
- ② 动态库：程序编译时不会链接到目标代码，在程序运行时载入，运行时需要动态库存在。动态库可方便多个程序共享一个函数库。后缀名为“.so”



GCC创建函数库

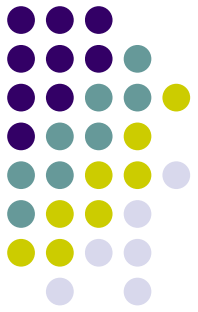
- 函数库的生成：由编译过的.o文件生成。
- 创建静态库：
 - 1.将需要生成函数库的函数执行gcc -c，生成.o文件

gcc -c hello.c

- 2.由.o文件创建静态库，静态库命名格式为：
lib静态库名.a

ar rcs libmyhello.a hello.o

创建静态库



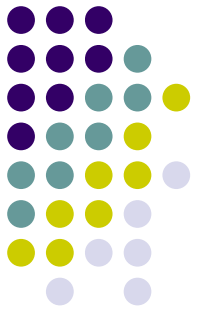
GCC创建函数库

- 3.使用静态库：在调用静态库的程序编译时指定静态库名

`$gcc -o hello main.c -L. -lmyhello`

`$./hello`

library 指定库
编译选项
链接库名



GCC创建函数库

- 创建动态库:

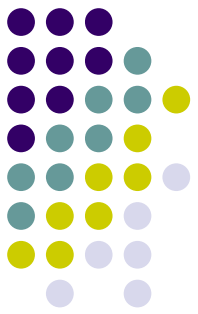
1. 由.o文件生成动态库, 动态库的命名: lib动态库名.so

```
gcc -shared -fPIC -o libmyhello.so hello.o
```

```
gcc -shared -fPIC -o libmyhello.so hello.c  
(centos版本)
```

centos -c → .o → .so

ubuntu -c → .so



GCC创建函数库

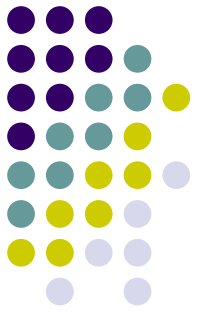
- 创建动态库：

2.使用动态库：用gcc命令指定动态库名进行编译，编译之前需将动态库文件复制到系统默认库函数目录 **/usr/lib** 中或者设置搜索路径。

gcc -o hello main.c -L. -lmyhello

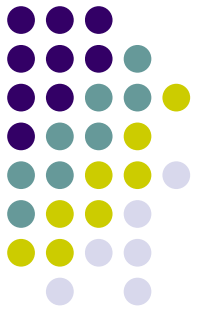
3.执行程序前设置（两种方式）：

- 把当前路径加入 **/etc/ld.so.conf** 中，执行运行 **ldconfig**
- **export** 把当前路径加入环境变量 **LD_LIBRARY_PATH**
export LD_LIBRARY_PATH=.:\$LD_LIBRARY_PATH



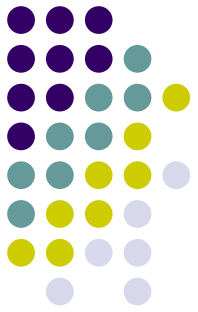
掌握的知识点

- gcc库函数目录和头文件目录指定
- 静态库和动态库函数的区别



Part 2 GDB调试工具

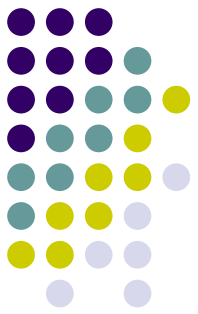
- GDB: GNU Debug
 - 设置断点
 - 监视变量值
 - 单步执行
 - 修改变量值



gdb commands

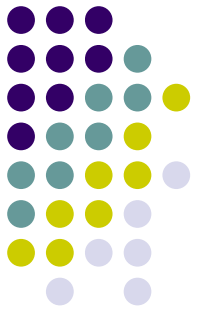
file	打开要调试的文件
break/tbreak	设置断点，可以是行号、函数名及地址(以*开头) tbreak : 设置临时断点
run	执行当前调试的程序
list	列出源代码的一部分
next	执行一条语句但不进入函数内部
step	执行一条语句，是函数则进入函数内部
display	显示表达式的值
print	临时显示表达式的值
kill	中止正在调试的程序
quit	退出 gdb
shell	不退出 gdb 就执行 shell 命令
make	不退出 gdb 就执行 make

Gdb using

A screenshot of a SecureCRT terminal window. The title bar reads "202.207.123.158 - SecureCRT". The menu bar includes "File", "Edit", "View", "Options", "Transfer", "Script", "Tools", "Window", and "Help". The toolbar contains various icons for file operations and terminal functions. The terminal text shows a user logging in as "wyc" on "localhost" and typing "gdb". The GDB startup banner follows, including version information and copyright notice. The prompt "(gdb)" is shown at the end of the output.

```
[wyc@localhost wyc]$ gdb
GNU gdb Asianux (6.0post-0.20040223.17.1AX)
Copyright 2004 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License.
You are welcome to change it and/or distribute copies of it under
certain conditions. Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for
details. This GDB was configured as "i386-asianux-linux-gnu".
(gdb) █
```

Ready Telnet 9, 7 16 Rows, 80 Cols VT100

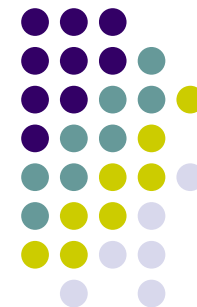


Gdb using(1)

- ✓ \$gdb filename
- ✓ gdb将装入名为filename的**可执行文件**。
- ✓ 在编译时需要使用**-g**选项

必须加上。

Gdb using(1)

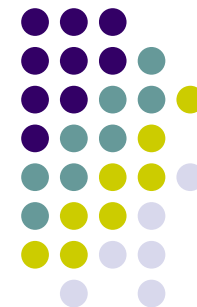


```
202.207.123.158 - SecureCRT
File Edit View Options Transfer Script Tools Window Help
[wyc@localhost wyc]$ gdb test
GNU gdb Asianux (6.0post 0.20040223.17.1AX)
Copyright 2004 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License.
You are welcome to change it and/or distribute copies of it under
certain conditions. Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for
details. This GDB was configured as "i386-asianux-linux-gnu"...Using
shared library "/lib/tls/libthread_db.so.1".

(gdb)
```

装入test文件

Gdb using(1)



202.207.123.158 - SecureCRT

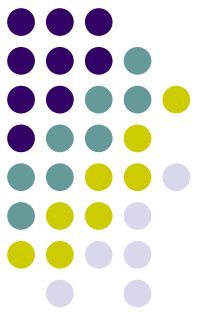
File Edit View Options Transfer Script Tools Window Help

public License, and you are
welcome to change it and/or distribute copies of it
under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show
warranty" for details.
This GDB was configured as "i386-asianux-linux-gnu
".
(gdb) file test
Reading symbols from test.+.done.
Using host libthread_db library "/lib/tls/libthrea
d_db.so.1".
(gdb)

Ready Telnet 13, 7 13 Rows, 50 Cols VT100

装入test文件

Gdb using(2) -list

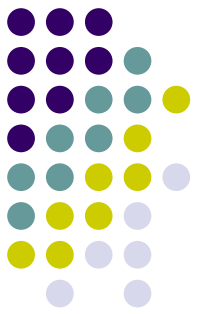


```
202.207.123.158 - SecureCRT
File Edit View Options Transfer Script Tools Window Help

There is absolutely no warranty for GDB.  Type "show war
This GDB was configured as "i386-asianux-linux-gnu"++.Us:
brary "/lib/tls/libthread_db.so.1".

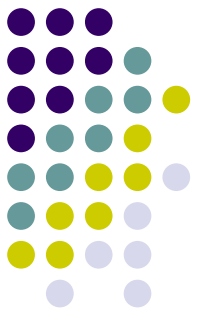
(gdb) list 1,10
1      #include<stdio.h>
2
3      main(int argc ,char *argv[])
4      {
5          char str[20];
6          int i;
7
8          if( 3 != argc)
9          {
10             printf("usage %s host port\n" ,a
(gdb)
```

Gdb using(2) -list



```
202.207.123.158 - SecureCRT
File Edit View Options Transfer Script Tools Window Help
7
8     if( 3 != argc)
9     {
10         printf("usage %s host port\n" ,a
(gdb) list
11         exit(1);
12     }
13
14     memset(str , '\0' ,sizeof(str));
15     strcpy(str,"This is a C program.");
16     printf("%s\n" , str);
17
18     for(i=0 ;i<argc ;i++)
19     {
20         printf("The no.%d argument is :%"
(gdb)
```

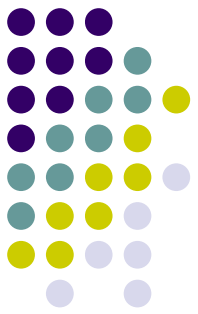
Gdb using(2)



```
(gdb) list
11             exit(1);
12         }
13
14         memset(str, '\0', sizeof(str));
15         strcpy(str, "This is a C program.");
16         printf("%s\n", str);
17
18         for(i=0 ;i<argc ;i++)
19         {
20             printf("The no.%d argument is :%",
(gdb) 回车符
21         }
22
23     }
(gdb)
```

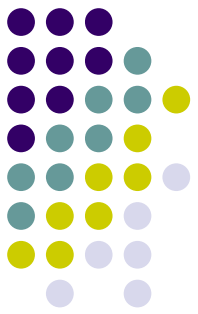
重复执行上一条命令

Gdb using(3)-help

A screenshot of a SecureCRT terminal window. The title bar reads "202.207.123.158 - SecureCRT". The menu bar includes "File", "Edit", "View", "Options", "Transfer", "Script", "Tools", "Window", and "Help". The toolbar contains various icons for file operations and debugging. The terminal text shows the GDB prompt "(gdb)", followed by the command "help" which is circled in red. Below this, it displays "List of classes of commands:" followed by a list of command categories and their descriptions. The status bar at the bottom shows "Ready", "Telnet", "15, 1", "15 Rows, 54 Cols", and "VT100".

```
(gdb)
(gdb) help
List of classes of commands:

aliases -- Aliases of other commands
breakpoints -- Making program stop at certain points
data -- Examining data
files -- Specifying and examining files
internals -- Maintenance commands
obscure -- Obscure features
running -- Running the program
stack -- Examining the stack
status -- Status inquiries
support -- Support facilities
tracepoints -- Tracing of program execution without st
```

Gdb using(4)-break

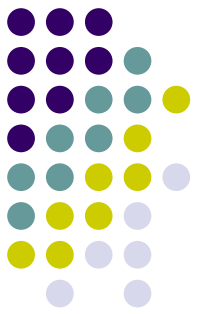
✓设置断点

```
202.207.123.158 - SecureCRT
File Edit View Options Transfer Script Tools Window Help
u are
welcome to change it and/or distribute copies of it unc
tions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show wa
ils.
This GDB was configured as "i386-asianux-linux-gnu"...l
ead_db library "/lib/tls/libthread_db.so.1".

(gdb) b 1
Breakpoint 1 at 0x80483e0: file test.c, line 1.
(gdb) break 1
Note: breakpoint 1 also set at pc 0x80483e0.
Breakpoint 2 at 0x80483e0: file test.c, line 1.
(gdb)
```

在第一行设置断点

Gdb using(5)-info breakpoints

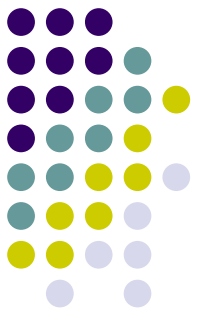


✓显示断点信息

```
202.207.123.158 - SecureCRT
File Edit View Options Transfer Script Tools Window Help
Breakpoint 1 at 0x80483e0: file test.c, line 1.
(gdb) info breakpoints
Num Type          Disp Enb Address      What
1  breakpoint      keep y   0x080483e0  in main
                                   at test.c:1
(gdb) b 10
Breakpoint 2 at 0x80483f6: file test.c, line 10.
(gdb) info breakpoints
Num Type          Disp Enb Address      What
1  breakpoint      keep y   0x080483e0  in main
                                   at test.c:1
2  breakpoint      keep y   0x080483f6  in main
                                   at test.c:10
(gdb) █
```

Ready Telnet 14, 7 14 Rows, 60 Cols VT100

Gdb using(6)-clear

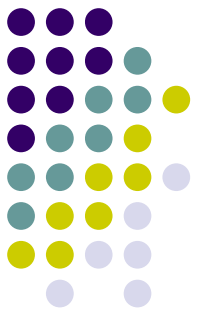


✓清除已经定义的断点

```
202.207.123.158 - SecureCRT
File Edit View Options Transfer Script Tools Window Help
Starting program: /home/wyc/test

Breakpoint 1, main (argc=1, argv=0xfeffdfd4) at test.c:10
10      printf("usage %s host port\n", argv[
0]);
(gdb) info breakpoints
Num Type           Disp Enb Address      What
1  breakpoint      keep y   0x080483f6 in main at test.c:10
      breakpoint already hit 1 time
(gdb) clear
Deleted breakpoint 1
(gdb) info breakpoints
No breakpoints or watchpoints.
(gdb)
```

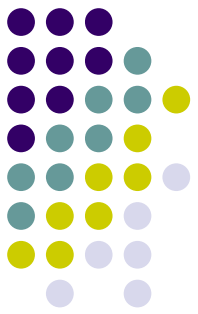
Ready Telnet 14, 7 14 Rows, 60 Cols VT100



Gdb using(7)-delete

✓ delete [bkpoints-num] 删除指定/全部断点

```
202.207.123.158 - SecureCRT
File Edit View Options Transfer Script Tools Window Help
Breakpoint 2 at 0x80483e0: file test.c, line 1.
(gdb) b 10
Breakpoint 3 at 0x80483f6: file test.c, line 10.
(gdb) info breakpoints
Num Type           Disp Enb Address      What
2   breakpoint      keep y   0x080483e0 in main at test.c:1
3   breakpoint      keep y   0x080483f6 in main at test.c:10
(gdb) de
define delete detach
(gdb) delete 3
(gdb) info breakpoints
Num Type           Disp Enb Address      What
2   breakpoint      keep y   0x080483e0 in main at test.c:1
(gdb) 
```

Gdb using(8)-print

✓ print [bkpoints-num] 输出断点处变量的值

```
202.207.123.158 - SecureCRT
File Edit View Options Transfer Script Tools Window Help

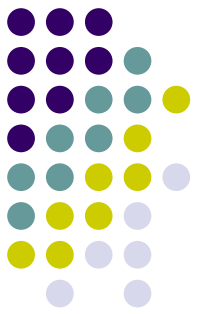
This GDB was configured as "i386-asianux-linux-gnu"...Using
host libthread_db library "/lib/tls/libthread_db.so.1".

(gdb) b 10
Breakpoint 1 at 0x80483f6: file test.c, line 10.
(gdb) run
Starting program: /home/wyc/test

Breakpoint 1, main (argc=1, argv=0xfeffa074) at test.c:10
10         printf("usage %s host port\n", argv[
0]);
(gdb) print argv[0]
$1 = 0xfeffcbb6 "/home/wyc/test"
(gdb)
```

Ready Telnet 14, 7 14 Rows, 60 Cols VT100

Gdb using(9)-quit

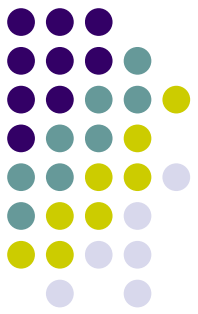


✓quit: 退出gdb

A screenshot of a SecureCRT terminal window. The title bar reads "202.207.123.158 - SecureCRT". The menu bar includes File, Edit, View, Options, Transfer, Script, Tools, Window, and Help. The toolbar contains various icons for file operations and terminal functions. The main text area shows the output of a GDB session. It starts with the GDB startup banner, followed by the user typing the 'quit' command, and ends with the shell prompt. The status bar at the bottom shows "Ready", "Telnet", "14, 22", "14 Rows, 60 Cols", and "VT100".

```
GNU gdb Asianux (6.0post-0.20040223.17.1AX)
Copyright 2004 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License,
and you are welcome to change it and/or distribute copies of it under
certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB.  Type "show warranty" for
details.
This GDB was configured as "i386-asianux-linux-gnu"...Using host
libthread_db library "/lib/tls/libthread_db.so.1".

(gdb) quit
[wyc@localhost wyc]$
```



Gdb 使用举例

- 源代码如下

```
// bug.c

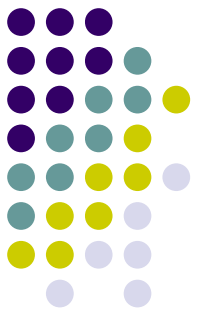
#include <stdio.h>
#include <stdlib.h>

static char buff[256];
static char* string;

int main()
{
    printf("input a string:");
    gets(string);

    printf("\n Your string is:%s\n",string);
    return 0;
}
```

编译：
gcc -o bug bug.c

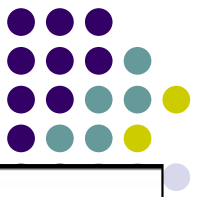


编译并运行

```
[root@donger gcctest]# ls
add.c  bug.c      minus.c  mytest.c  test.c  ts
add.o  gcctest.c  minus.o  test      test.o
[root@donger gcctest]# gcc -o bug bug.c 编译
/tmp/ccgab33e.o(.text+0x36): In function `main':
bug.c: warning: the `gets' function is dangerous
and should not be used.
[root@donger gcctest]# ls
add.c  bug.c      minus.o  test.c
add.o  gcctest.c  mytest.c  test.o
bug    minus.c    test     ts
[root@donger gcctest]# ./bug
input a string:hello
段错误
[root@donger gcctest]#
```

?? ??

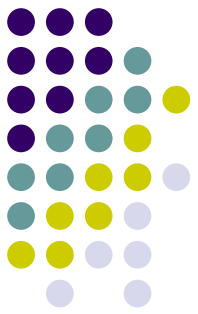
使用gdb调试bug



```
[root@donger gcctest]# gdb bug
GNU gdb Red Hat Linux (6.3.0.0-1.21rh)
Copyright 2004 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty" for details.
This GDB was configured as "i386-redhat-linux-gnu"...
(no debugging symbols found)
Using host libthread_db library "/lib/libthread_db.so.1".

(gdb) run 运行bug
Starting program: /home/donger/gcctest/bug
Reading symbols from shared object read from target memory...(no debugging sy
mbols found)...done.
Loaded system supplied DSO at 0xeac000
(no debugging symbols found)
(no debugging symbols found)
input a string:hello 输入字符串hello

Program received signal SIGSEGV, Segmentation fault.
0x002647e0 in 显示出错位置 /lib/libc.so.6
(gdb) where
#0 0x002647e0 in gets () from /lib/libc.so.6
#1 0x080483ea in main ()
(gdb) █ 能不能看到源代码呢?
```



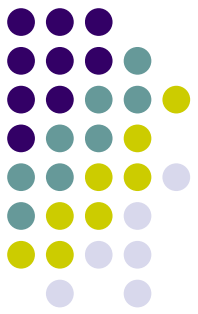
使用gcc的-g参数

- gcc **-g** -o bug bug.c
- 重新调试

```
Program received signal SIGSEGV, Segmentation fault.
0x002647e0 in gets () from /lib/libc.so.6
(gdb) where
#0  0x002647e0 in gets () from /lib/libc.so.6
#1  0x080483ea in main () at bug.c:12
(gdb) list
12      gets(string);
13
14      printf("\n Your string is:%s\n",string);
15      return 0;
16  }
```

(gdb) █

列出源代码



设置断点

```
(gdb) break 12
```

```
Breakpoint 2 at 0x80483dc: file bug.c, line 12.
```

```
(gdb) next
```

```
Single stepping until exit from function gets,  
which has no line number information.
```

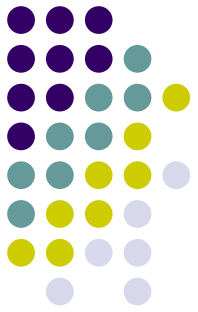
```
Program terminated with signal SIGSEGV, Segmentation fault.  
The program no longer exists.
```

```
(gdb) print string
```

```
$1 = 0x0
```

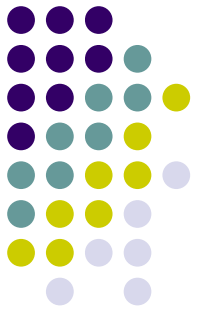
```
(gdb) █
```

? 怎么修改前面的源代码呢? (作业)



调试子命令总结

- **file** : 装入想要调试的可执行文件
- **kill**: 终止正在调试的程序
- **list**: 列出正在执行的程序清单
- **next**: 执行一行代码但不进入函数内部
- **step**: 执行一行代码并进入函数内部
- **run**: 执行当前正在调试的程序
- **quit**: 终止gdb调试
- **break**: 设置断点 (**break** 行号)
- **watch**: 设置观察点, 观察表达式的值是否发生变化



- **info: 查看断点信息**

info breakpoints、**info watchpoints**

info break 显示当前断点清单，包括到达断点处的次数等。

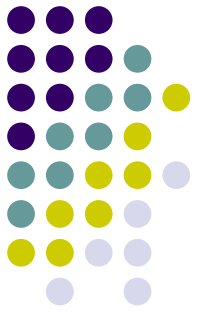
info files 显示被调试文件的详细信息。

info func 显示所有的函数名称。

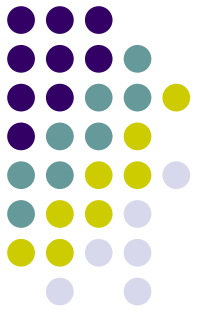
info local 显示当函数中的局部变量信息。

info prog 显示被调试程序的执行状态。

info var 显示所有的全局和静态变量名称



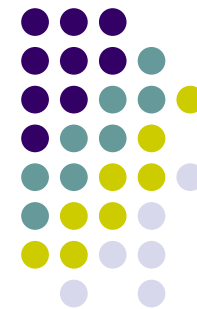
- **delete:** 删除某个或所有的断点
delete 断点号 或 delete
- **disable:** 使断点失效（但仍存在）
- **enable:** 使断点有效
- **clear:** 清除断点信息
clear 断点所在行号
clear 函数入口
- **continue:** 继续执行程序直到程序结束



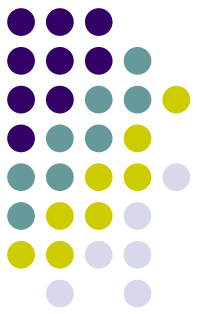
掌握的知识点

- Gdb程序调试方法
- Gdb常用命令

Part 3 Make工具



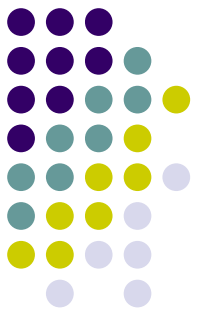
- Make引入
- Makefile文件



3.1 Make 引入

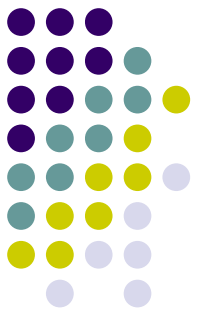
✓Make的引入:

- 文件数量太大, 手工gcc编译不方便
- 仅需要编译已经做了修改的源代码文件; 其他文件只需要重新连接
- 记录哪些文件已改变且需要编译, 哪些文件仅仅需要连接很难



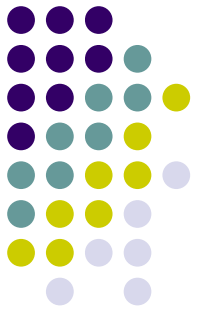
3.2 make & makefile

- Multi-file project
 - IDE---Eclipse
 - make
- make & makefile
 - makefile描述模块间的依赖关系:
 - make命令根据makefile对程序进行管理和维护;
make判断被维护文件的时序关系



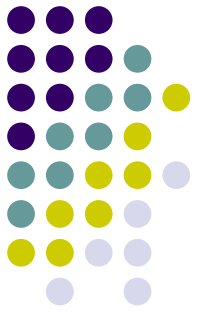
make

- `make [-f filename] [targetname]`
- 使用方法:
 - ❖ **make** 自动找当前目录下名为 **Makefile/makefile** 的文件
 - ❖ **make -f** 文件名 找当前目录下指定文件名的文件



makefile的组成

- 显式规则：明确指出目标文件的生成规则
- 隐式规则：需要**make**自动推导的规则
- 变量定义：声明时赋值，引用时加"\$"
object = main.o
- 文件指示：引用外部的文件
- 注释：#

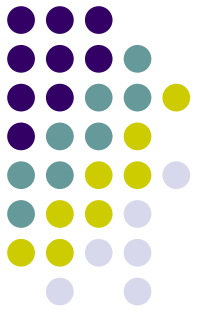


Makefile的显式规则

- 规则

一条规则包含3个方面的内容，

- 1) 要创建的目标（文件），
- 2) 创建目标（文件）所依赖的文件列表；
- 3) 通过依赖文件创建目标文件的命令组



Makefile的显式规则

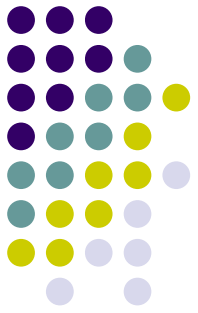
- 规则一般形式

target ... : prerequisites ...

<tab>command

<tab>...

<tab>...



Makefile的显式规则

✓ 每条规则由一个带冒号的“依赖行”和一条或多条以 **tab** 开头的“命令行”组成

目标1 [目标2...]:[依赖文件列表]

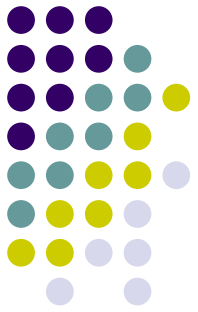
[\t 命令]

...

ex:

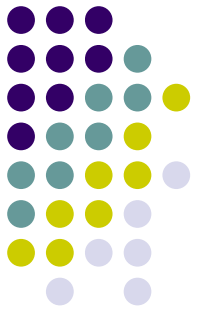
make_test: make_main.o wrtlog.o

<TAB> gcc -o make_test make_main.o wrtlog.o



Makefile的显式规则

- ✓冒号左边是目标,冒号右边是依赖文件
- ✓目标和依赖文件均是由字母、数字、句点和斜杠组成的字符串
- ✓目标或依赖文件的数目多于一个时,以空格分隔



Makefile举例

make_test:make_main.o wrtlog.o

gcc -o make_test make_main.o wrtlog.o

make_main.o:make_main.c

gcc -o make_main.o -c make_main.c

wrtlog.o:wrtlog.c

gcc -o wrtlog.o -c wrtlog.c

编译目标

2023-6-17

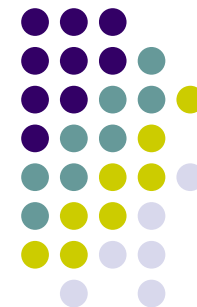
命令

青岛科技大学信息科学技术学院

依赖文件

60

Makefile



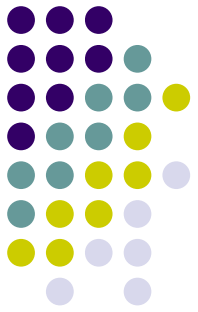
The screenshot shows a SecureCRT terminal window with the title bar '202.207.123.151 - SecureCRT'. The menu bar includes 'File', 'Edit', 'View', 'Options', 'Transfer', 'Script', 'Tools', 'Window', and 'Help'. The toolbar contains various icons for file operations and terminal control. The terminal content shows a user at the prompt '[oracle@localhost c_program]#' editing a file named 'makefile' with the command 'vi makefile'. The Makefile content is as follows:

```
[oracle@localhost c_program]# vi makefile
make_test:make_main.o wrtlog.o
    gcc -o make_test make_main.o wrtlog.o
make_main.o:make_main.c
    gcc -o make_main.o -c make_main.c
wrtlog.o:wrtlog.c
    gcc -o wrtlog.o -c wrtlog.c
```

The status bar at the bottom indicates 'Ready', 'Telnet', '1, 1', '18 Rows, 77 Cols', and 'VT100'.

一个简单的makefile

分析程序文件间的联系

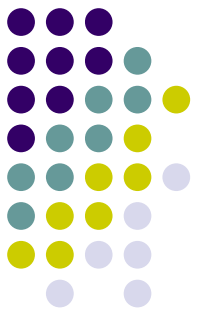


```
edit : main.o kbd.o command.o display.o insert.o search.o files.o utils.o
    gcc -o edit main.o kbd.o command.o display.o insert.o \
        search.o files.o utils.o
main.o : main.c defs.h
    gcc -c main.c
kbd.o : kbd.c defs.h command.h
    gcc -c kbd.c
command.o : command.c defs.h command.h
    gcc -c command.c
display.o : display.c defs.h buffer.h
    gcc -c display.c
insert.o : insert.c defs.h buffer.h
    gcc -c insert.c
search.o : search.c defs.h buffer.h
    gcc -c search.c
files.o : files.c defs.h buffer.h command.h
    gcc -c files.c
utils.o : utils.c defs.h
    gcc -c utils.c
```

清除目标文件

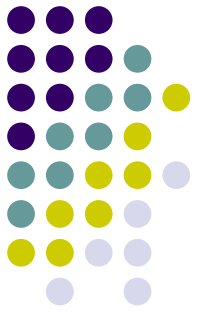
clean:

```
rm edit main.o kbd.o command.o display.o insert.o search.o files.o utils.o
```

Make的工作过程

- default goal
在缺省的情况下，make从makefile中的第一个目标开始执行
- Make的工作过程类似一次深度优先遍历过程



Makefile的变量

- 自定义变量
例：

```
object=main.o Add.o Sub.o Mul.o Div.o
```

```
exe : $(object)
```

```
gcc -o exe $(object)
```

- 特殊变量

\$@：表示目标文件；

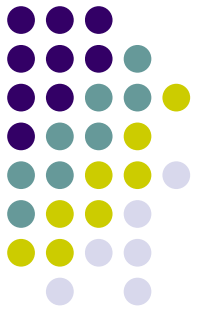
\$^：表示所有依赖目标的集合，以空格分隔；

\$<：表示第一个依赖文件；

\$@ 所有目标！

\$^ 所有依赖

\$< 第一个依赖



Makefile的隐式规则（自动推导）

- **make**能够自动推导文件以及文件依赖关系后面的命令

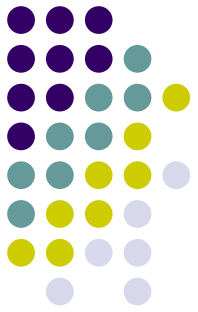
例：

```
object=main.o Add.o Sub.o Mul.o Div.o
```

```
exe : $(object)
```

```
gcc -o $@ $(object)
```

```
main.o : def.h
```



Makefile的伪目标

- makefile使用.PHONY关键字来定义一个伪目标，具体格式为：

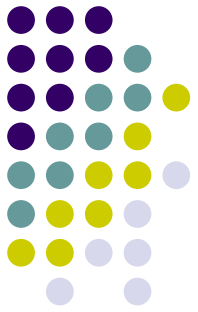
.PHONY : 伪目标名称

例：

.PHONY : clean 伪目标名称

clean :

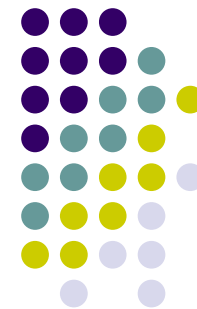
rm \$(object)



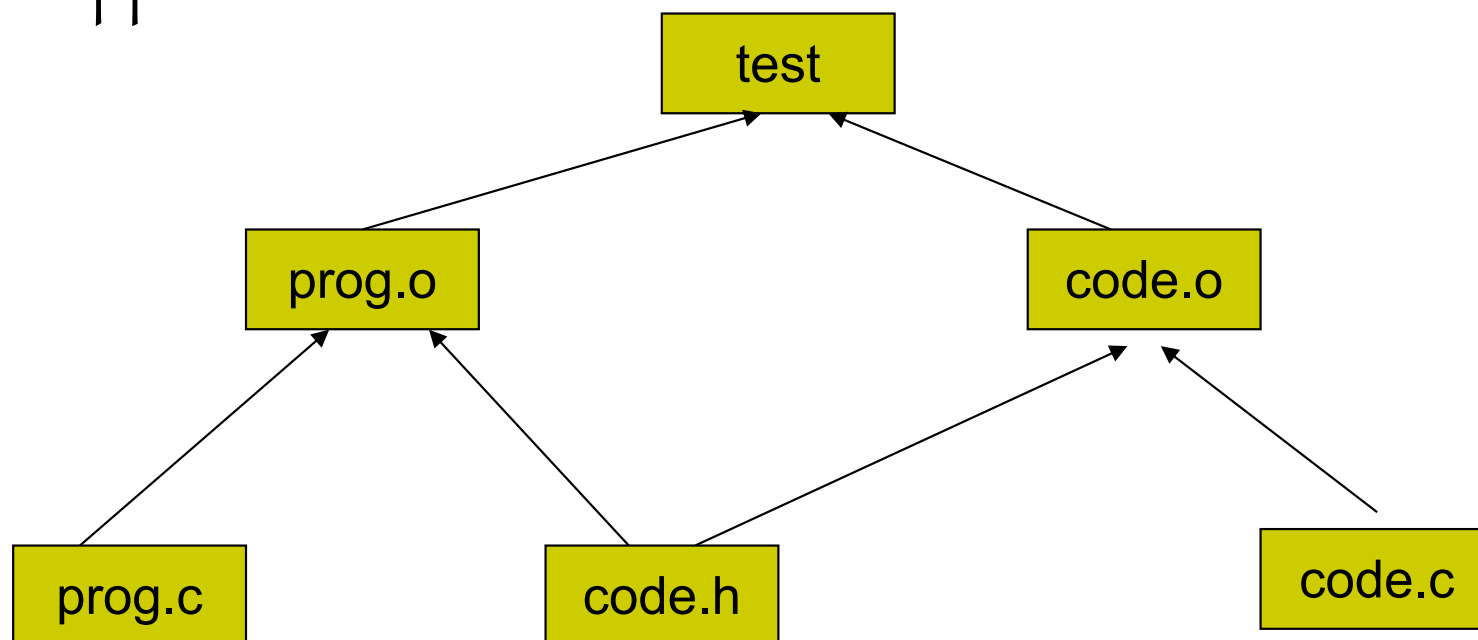
掌握的知识点

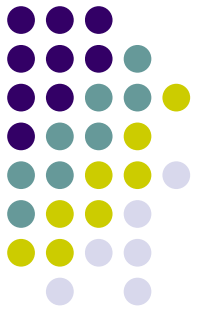
- Make作用
- Make命令使用方法
- Makefile显式书写规则

练习



- 根据以下函数之间的依赖关系写出makefile文件





总结: C语言编程步骤

- 编辑: vi ,emacs,gedit,Eclipse...
- 编译: gcc
- 调试: gdb
- 运行: ./executable file
- 项目管理: make